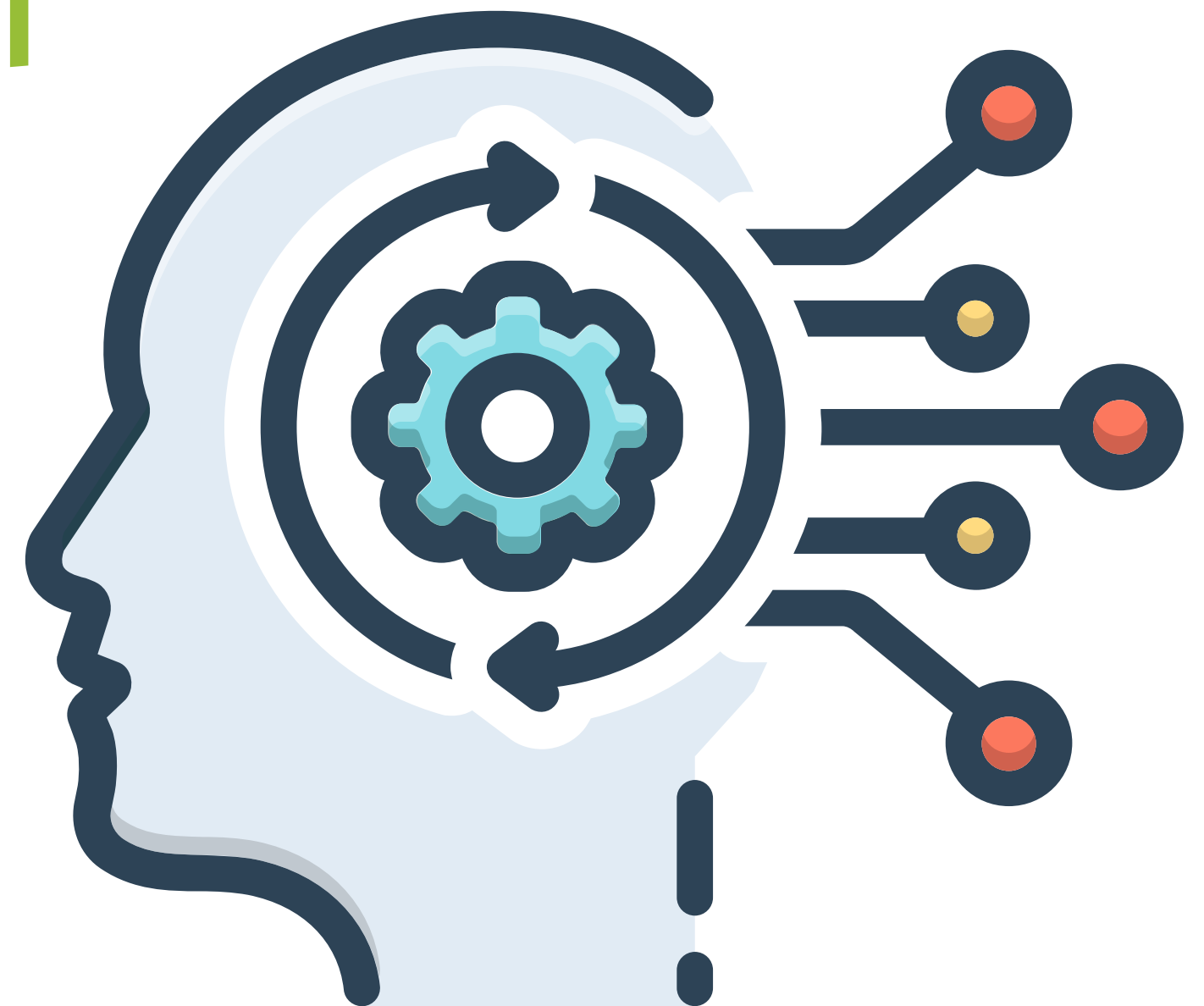


Algorithmes et Programmation II

Les fichiers en C

INFO – AGROTIC

Last update: january 2026



Sommaire



- Introduction et concepts de base
- Ouverture et fermeture de fichier
- Lecture de fichiers
- Ecriture de fichiers
- Exemples et positionnement
- Bonnes pratiques

Introduction et Concepts de Base (1/2)

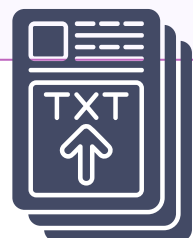
Qu'est-ce qu'un fichier en C ?

Un fichier est un flux de données stocké sur un support de stockage persistant (disque dur, SSD).
En C, on manipule les fichiers via des pointeurs de type **FILE***.

Types de fichiers

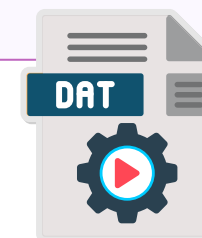
Fichiers texte :

- Contenu lisible (caractères ASCII/UTF-8)
- Fin de ligne gérée automatiquement
- **Exemple** : .txt, .csv, .c



Fichiers binaires :

- Contenu en octets bruts
- Manipulation directe de la mémoire
- **Exemple** : .bin, .dat, images, exécutables



Introduction et Concepts de Base (2/2)

C'est une structure opaque qui contient toutes les informations nécessaires pour manipuler un fichier.

```
FILE *fichier; // Déclaration d'un pointeur de fichier
```

Ouverture et Fermeture de Fichiers (1/2)

Fonction **fopen()**

```
FILE *fopen(const char *nom_fichier, const char *mode);
```

Modes d'ouverture principaux :

Mode	Description	Fichier existe	Fichier n'existe pas
"r"	Lecture seule	Ouverture OK	Erreur (NULL)
"w"	Écriture (écrase)	Écrase le contenu	Crée le fichier
"a"	Ajout (append)	Ajoute à la fin	Crée le fichier
"r+"	Lecture/Écriture	Ouverture OK	Erreur (NULL)
"w+"	Lecture/Écriture	Écrase le contenu	Crée le fichier

Modes binaires :

Ajouter b - exemple :

- "rb",
- "wb",
- "ab"

Ouverture et Fermeture de Fichiers (2/2)

Exemple complet

```
FILE *fichier = fopen("donnees.txt", "r");  
if (fichier == NULL) {  
    perror("Erreur d'ouverture");  
    return 1;  
}  
  
// ... opérations sur le fichier ...  
fclose(fichier); // TOUJOURS fermer le fichier !
```

Fonction **fclose()**

```
int fclose(FILE *fichier);
```



Libère les ressources et vide les tampons. Retourne 0 si succès, EOF sinon.

Lecture dans les Fichiers (1/4)

Lecture caractère par caractère

```
int fgetc(FILE *fichier); // Retourne le caractère ou EOF
```

Exemple :

```
int c;  
while ((c = fgetc(fichier)) != EOF) {  
    putchar(c);  
}
```

Lecture dans les Fichiers (2/4)

Lecture ligne par ligne

```
char *fgets(char *buffer, int taille, FILE *fichier);
```

Lit au maximum `taille-1` caractères ou jusqu'à `'\n'`.
Retourne **NULL** en fin de fichier.

Exemple :

```
char ligne[256];  
while (fgets(ligne, sizeof(ligne), fichier) != NULL) {  
    printf("%s", ligne);  
}
```


Lecture dans les Fichiers (3/4)

Lecture formatée

```
int fscanf(FILE *fichier, const char *format, ...);
```

Exemple :

```
int age;  
char nom[50];  
fscanf(fichier, "%s %d", nom, &age);
```

Lecture dans les Fichiers (4/4)

Lecture binaire

```
size_t fread(void *ptr, size_t taille, size_t nombre, FILE *fichier);
```

Lit **nombre** éléments de taille octets.
Retourne le nombre d'éléments lus.

Exemple :

```
int tableau[10];  
fread(tableau, sizeof(int), 10, fichier);
```

Écriture dans les Fichiers (1/4)

Écriture caractère par caractère

```
int fputc(int caractere, FILE *fichier);
```

Exemple :

```
fputc('A', fichier);
```

Écriture de chaînes

```
int fputs(const char *chaine, FILE *fichier);
```



**N'ajoute PAS
automatiquement
le '\n'.**

Écriture dans les Fichiers (2/4)

Écriture caractère par caractère

```
int fputc(int caractere, FILE *fichier);
```

Exemple :

```
fputc('A', fichier);
```

Écriture de chaînes

```
int fputs(const char *chaine, FILE *fichier);
```

↘

```
fputs("Bonjour\n", fichier);
```



N'ajoute PAS
automatiquement
le '\n'.

Écriture dans les Fichiers (3/4)

Écriture formatée

```
int fprintf(FILE *fichier, const char *format, ...);
```

Identique à printf() mais écrit dans un fichier.

Exemple :

```
fprintf(fichier, "Nom: %s, Age: %d\n", nom, age);
```


Écriture dans les Fichiers (4/4)

Écriture binaire

```
size_t fwrite(const void *ptr, size_t taille, size_t nombre, FILE *fichier);
```

Exemple :

```
int tableau[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
fwrite(tableau, sizeof(int), 10, fichier);
```

Exemple complet

Copie de fichiers

```
FILE *source = fopen("original.txt", "r");
FILE *dest = fopen("copie.txt", "w");
int c;
while ((c = fgetc(source)) != EOF) {
    fputc(c, dest);
}
fclose(source);
fclose(dest);
```

Positionnement et Fonctions Utiles

Déplacement dans le fichier

- **fseek()** - Positionner le curseur

```
int fseek(FILE *fichier, long offset, int origine);
```

Origines possibles :

- **SEEK_SET** : Début du fichier
- **SEEK_CUR** : Position actuelle
- **SEEK_END** : Fin du fichier

Positionnement et Fonctions Utiles

Déplacement dans le fichier

- **fseek()** - Positionner le curseur

```
int fseek(FILE *fichier, long offset, int origine);
```

Exemples :

```
fseek(fichier, 0, SEEK_SET);    // Retour au début
fseek(fichier, 10, SEEK_CUR);   // Avance de 10 octets
fseek(fichier, -5, SEEK_END);   // 5 octets avant la fin
```

Positionnement et Fonctions Utiles

Déplacement dans le fichier

- **ftell()** - Obtenir la position

```
long ftell(FILE *fichier); // Retourne la position ou -1L
```

- **rewind()** - Retour au début

```
void rewind(FILE *fichier); // Équivalent à fseek(f, 0, SEEK_SET)
```

- Détection de fin de fichier et **erreurs**

```
int feof(FILE *fichier); // Retourne non-zéro si fin de fichier
```

```
int ferror(FILE *fichier); // Retourne non-zéro si erreur
```

```
void clearerr(FILE *fichier); // Réinitialise les indicateurs
```

Taille d'un fichier

```
fseek(fichier, 0, SEEK_END);  
long taille = ftell(fichier);  
rewind(fichier);  
printf("Taille : %ld octets\n", taille);
```

Exemple complet de résumé

```
FILE *f = fopen("data.txt", "r");  
if (f == NULL) {  
    perror("Erreur ouverture");  
    return 1;  
}  
  
char ligne[100];  
while (fgets(ligne, sizeof(ligne), f) != NULL) {  
    printf("%s", ligne);  
}  
  
if (ferror(f)) {  
    perror("Erreur lecture");  
}  
  
fclose(f);
```



**Ouverture de fichier
"data.txt" en mode
lecture**

Exemple complet de résumé

```
FILE *f = fopen("data.txt", "r");  
if (f == NULL) {  
    perror("Erreur ouverture");  
    return 1;  
}
```

```
char ligne[100];  
while (fgets(ligne, sizeof(ligne), f) != NULL) {  
    printf("%s", ligne);  
}
```

```
if (ferror(f)) {  
    perror("Erreur lecture");  
}
```

```
fclose(f);
```



**Lecture par ligne de
100 caractères
chacun**

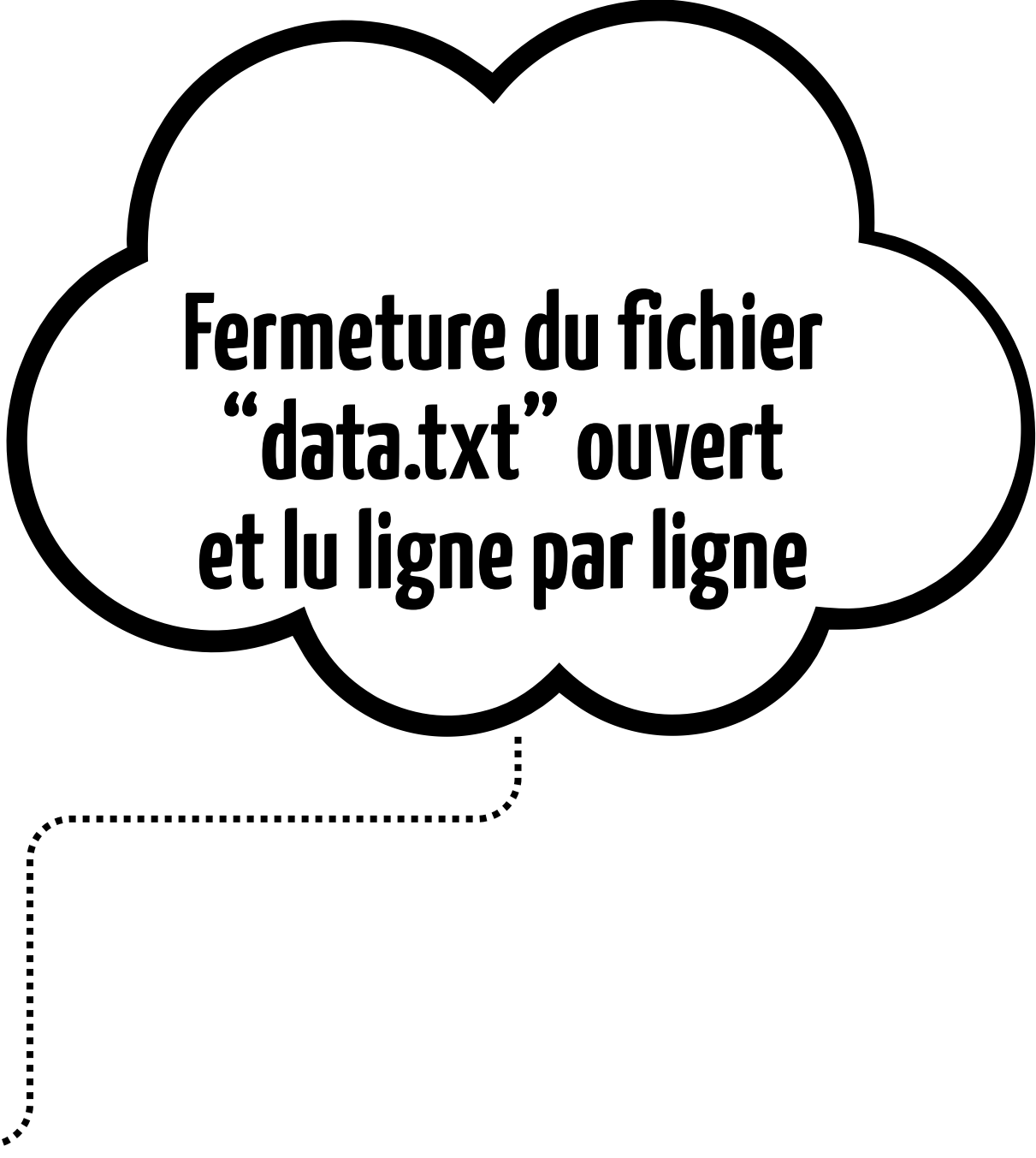
Exemple complet : lecture sécurisée d'un fichier

```
FILE *f = fopen("data.txt", "r");
if (f == NULL) {
    perror("Erreur ouverture");
    return 1;
}

char ligne[100];
while (fgets(ligne, sizeof(ligne), f) != NULL) {
    printf("%s", ligne);
}

if (ferror(f)) {
    perror("Erreur lecture");
}

fclose(f);
```



Fermeture du fichier
"data.txt" ouvert
et lu ligne par ligne

Bonnes pratiques

✓ À FAIRE :

- Toujours vérifier si `fopen()` retourne `NULL`
- Fermer les fichiers avec `fclose()`
- Vérifier les erreurs avec `feof()` et `ferror()`
- Utiliser `perror()` pour afficher les erreurs

✗ À ÉVITER :

- Oublier de fermer les fichiers (fuite de ressources)
- Ne pas vérifier les valeurs de retour
- Mélanger lecture binaire et texte sans précaution
- Dépasser la taille des buffers